Deep Dive Page — Information Document

Page Overview

Purpose

The Deep Dive page (route `/dive`, rendered by `DeepDivePage.tsx`) runs a **targeted one-off query against a single repository**. It is the narrow counterpart to Deep Sweep: same pipeline, constrained to one source, and optimised for exhaustively covering that source for a focused topic. The page header reads `Deep Dive` with the subtitle `Targeted single-repository query`.

Primary User Flows

The page supports three flows, each driven entirely from the single form on the page:

1. **Launch a dive.** Pick a repository, optionally restrict the date range, enter one or more keywords, adjust the `Max Results` slider, optionally toggle `Enable AI Summarization` (and, under the disclosure, override length/tone/model for this run only), then click `Run Deep Dive`. The page `POST`s to `/api/search/dive`, captures the returned `execution_id`, stores it in `pageExecIdRef`, and calls `startExecution(id, 'deep_dive', [repository])` on `ExecutionContext` so the floating widget and Monitor page begin streaming progress.
2. **Reuse a saved configuration.** The inline `ConfigLoader` (`configType="manual_dive"`) lets the user load a previously saved `manual_dive` row; `applyConfig` restores `repository`, `keywords`, `max_results`, and `ai_enabled` (but deliberately not the date range).
3. **Save the current form as a configuration.** Clicking `Save Configuration` opens a modal that `POST`s to `/api/configurations` with `config_type: "manual_dive"` and a `parameters` object containing `repository`, `date_from`, `date_to`, `keywords`, `max_results`, and `ai_enabled`. A success toast (`Configuration saved.`) auto-dismisses after 3 seconds.

A post-run **Execution #N** result card is rendered inline (outside the form) once `pageExec` reaches a terminal state (not `running` and not `cancelling`), showing status, result count, new count, elapsed seconds, and a `View Report` anchor to `#/results?exec=<id>`.

Inputs and Outputs

Inputs (observed in `DeepDivePage.tsx` and `resmon.py`):

- `useExecution()` → `{ activeExecutions, startExecution }` — active-execution registry and dispatcher.
- `useRepoCatalog()` → `{ bySlug, presence, refreshPresence }` — repository catalog (from `/api/repositories/catalog`) and credential presence (from `/api/credentials`), used to decide whether to render `RepoKeyStatus` and whether a key is `present`.
- `ConfigLoader` (`configType="manual_dive"`) — fetches matching `saved_configurations` rows and invokes `applyConfig` when the user loads one.

Outputs:

- `POST /api/search/dive` with body `{ repository, query, keywords, date_from, date_to, max_results, ai_enabled, ephemeral_credentials, ai_settings?, saved_configurati`. Returns `{ execution_id }`. The optional `saved_configuration_id` is set from the page's `loadedConfigIdRef` whenever the user launched the run from a `ConfigLoader`-restored saved config; the backend persists it on the resulting `executions` row so the Recent Activity / Results & Logs / Calendar surfaces can render the matching saved-config name.
- `POST /api/configurations` with `{ name, config_type: "manual_dive", parameters: { ... } }` when saving.
- `ExecutionContext.startExecution(id, 'deep_dive', [repository])` — attaches the new execution's SSE stream to the global context.

Known Constraints or Permissions

- **Single repository only.** `RepositorySelector` is mounted with `mode="single"`; `handleRun` blocks submission with `"Please select a repository."` if no repo is chosen.
- **At least one keyword required.** `handleRun` blocks submission with `"Please enter at least one keyword."` if `keywords.length === 0`.
- **Max Results slider range.** The `<input type="range">` is bounded `min=10 max=500 step=10` with a default of `100`.
- **Date range is not saved.** `applyConfig` intentionally skips `date_from` / `date_to`, and the in-code comment confirms users must pick the date range fresh per run.

- **Credential handling.** If the chosen repository has a `credential_name` in the catalog and that name is already `present` in the OS keyring, the dive can run without any extra input. Otherwise `RepoKeyStatus` (`variant="dive"`) offers an ephemeral-key field whose value is threaded through `ephemeralKeys[name]` and sent only in the request body — never persisted.
- **AI overrides are opt-in and non-destructive.** The `Override AI settings for this run` disclosure is rendered only when `aiEnabled` is true. It mounts the shared `AIOverridePanel` component, which exposes the full Settings → AI control set (`Provider` , `Model` , `Length` , `Tone` , `Temperature` , `Extraction Goals`) with per-field merge semantics: each field defaults to empty (`"Use app default"`) and `buildAIOverridePayload` drops empty values before the request body is built, so the backend per-field merge never sees a blank that could clobber persisted defaults. The `Model` field is a provider-aware dropdown populated by `Load models` (which calls `POST /api/ai/models` for the chosen provider); when the chosen provider has no stored API key the panel renders an inline key input that POSTs to `/api/credentials/{name}` , and a `Save as default model` button persists the chosen model into `ai_default_models[provider]` without leaving the page.
- **Admission-controller cap.** The backend rejects the request with HTTP `429` and `Retry-After: 5` when resmon is already running the maximum number of concurrent executions (`admission.max()`); the detail message points users to `Settings → Advanced` . The page surfaces the server error verbatim via `setError` .
- **Form lifecycle.** While `pageExecIdRef.current` points at a running execution, the primary submit button re-labels to `Run Another` (otherwise `Run Deep Dive`). The `useEffect` keyed on `pageExec?.status` clears `pageExecIdRef` as soon as the execution reaches a terminal state, re-enabling the form without a page reload.

Frontend

Route and Main Component

- **Route:** `/dive` (registered in `App.tsx` ; the floating-widget "open this page" links use `#/dive`).
- **Main component:** `DeepDivePage` (default export of `frontend/src/pages/DeepDivePage.tsx`).

Child Components and Hooks

Imported and used directly by `DeepDivePage` :

- `PageHelp` (`storageKey="deep-dive"`) — collapsible help block with two sections, *When to use this page* and *How to use it*.
- `ConfigLoader` — loads saved `manual_dive` configurations and calls `applyConfig(parameters)` on selection. `refreshKey={configRefresh}` forces it to re-fetch after a successful save.
- `RepositorySelector` (`mode="single"`) — renders the repository dropdown.
- `KeywordCombinationBanner` — mounted under the selector when a repository is chosen; surfaces the upstream API's keyword-combination semantics (e.g. "Implicit AND", "Relevance-ranked") for the selected repository, with a tooltip pointing to the consolidated glossary on the Repositories & API Keys page.
- `RepoKeyStatus` (`variant="dive"`) — conditionally rendered when `repository && bySlug[repository]` ; surfaces a present/missing badge and an ephemeral-key input.
- `DateRangePicker` — two date inputs bound to `dateFrom` / `dateTo` .
- `KeywordInput` — chip-style keyword entry bound to the `keywords` array.
- `InfoTooltip` — hover tooltips on the `Max Results` and `Enable AI Summarization` labels.

Hooks:

- `useState` for `ephemeralKeys` , `repository` , `dateFrom` , `dateTo` , `keywords` , `maxResults` , `aiEnabled` , `aiOverrideLength` , `aiOverrideTone` , `aiOverrideModel` , `error` , `saveModalOpen` , `configName` , `saveStatus` , `configRefresh` .
- `React.useRef<number | null>` for `pageExecIdRef` — identifies the execution this page started, without re-rendering when it changes.
- `React.useEffect` keyed on `pageExec?.status` — clears `pageExecIdRef.current` once the execution leaves `running` / `cancelling` .
- `useExecution()` — returns `{ activeExecutions, startExecution }` from `ExecutionContext` .
- `useRepoCatalog()` — returns `{ bySlug, presence, refreshPresence }` .

UI State Model

Name	Type	Role
`repository`	`string`	Slug of the selected repository.
`dateFrom` , `dateTo`	`string`	Optional ISO date strings; sent as `null` when empty.
`keywords`	`string[]`	Chip-style keyword list. `buildQuery()` joins with spaces to form `query` .

Name	Type	Role
`maxResults`	`number`	Bounded 10–500, default 100.
`aiEnabled`	`boolean`	Gates both the `Override AI settings` disclosure and the `ai_enabled` request field.
`aiOverrideLength`	`string`	One of `""` , `brief` , `standard` , `detailed` .
`aiOverrideTone`	`string`	One of `""` , `technical` , `neutral` , `accessible` .
`aiOverrideModel`	`string`	Free-text model ID; trimmed before send.
`ephemeralKeys`	`Record<string, string>`	Per-credential-name ephemeral key values; empty values are stripped before send.
`error`	`string`	Inline form error.
`saveModalOpen` , `configName` , `saveStatus`	modal state	Drive the `Save Configuration` modal and its success/error banner.
`configRefresh`	`number`	Increments after a successful save to force `ConfigLoader` to re-fetch.
`pageExecIdRef`	`useRef<number`	null>`

Derived values:

- `pageExec = pageExecIdRef.current !== null ? activeExecutions[pageExecIdRef.current] : undefined` .
- `running = pageExecIdRef.current !== null && activeExecutions[pageExecIdRef.current]?.status === 'running'` — flips the submit label between `Run Deep Dive` and `Run Another` .

Key Interactions and Events

- **Form submit (`handleRun`).**
  i. Validate `repository` and `keywords` ; set `error` and abort if either is empty.
  ii. Build an `overrides` object from the AI-override inputs, dropping empty values (`length` , `tone` only added when truthy; `model` added only when `trim().length > 0`).
  iii. Assemble the request `body` with `repository` , `query: keywords.join(' ')` , `keywords` , `date_from: dateFrom || null` , `date_to: dateTo || null` , `max_results` , `ai_enabled` , and `ephemeral_credentials` (entries with whitespace-only values stripped via `Object.fromEntries(... .filter([,v] => v.trim().length > 0))`). If `overrides` has any keys, attach it as `body.ai_settings` .
  iv. `apiClient.post<{ execution_id: number }>('/api/search/dive', body)` .
  v. On success: `pageExecIdRef.current = execution_id` and `startExecution(execution_id, 'deep_dive', [repository])` .
  vi. On failure: `setError(err.message || 'Failed to start dive.')` — this is where the admission-controller `429` message surfaces.
- **Load configuration (`applyConfig`).** Restores `repository` , `keywords` , `max_results` , `ai_enabled` from the saved parameters; does not touch `date_from` / `date_to` .
- **Save configuration (`handleSaveConfig`).** Requires a non-empty trimmed `configName` . Posts to `/api/configurations` , closes the modal, clears `configName` , bumps `configRefresh` , and shows `Configuration saved.` for 3 seconds; errors surface as `Error: <message>` in the same banner.
- **Ephemeral-key change.** Writes into `ephemeralKeys[credential_name]` ; `RepoKeyStatus.onPresenceChange` calls `refreshPresence()` after the user stores a key through its embedded controls so the `present` badge updates without a page reload.
- **Post-run result card.** Only rendered when `pageExec` exists and its status is neither `running` nor `cancelling` . Styles switch between `.result-success` and `.result-failure` based on `pageExec.status === 'completed'` .

Error and Empty States

- **Validation errors.** Inline `.form-error` block under the form body shows `"Please select a repository."` or `"Please enter at least one keyword."` before the request is sent.
- **API errors.** The thrown error message from `apiClient.post` is rendered in the same `.form-error` block. Admission-controller rejections (HTTP `429`) show the server-supplied message instructing the user to wait or raise the cap in `Settings → Advanced` .
- **Save-configuration errors.** Appear as `Error: <message>` inside the `.form-success` banner (the banner is reused for both success and error messaging via `saveStatus`).
- **Empty result card.** Before any run (`pageExec` undefined) no result card is rendered; the post-run card only appears after a terminal status.

- **Missing credential.** When the chosen repository has a `credential_name` and `presence[name].present` is false, `RepoKeyStatus` displays a warning state and exposes the ephemeral-key input; the dive can still proceed if a key is typed into that input for the current run.

Backend

API Endpoints

The Deep Dive page exercises these FastAPI routes (all in `resmon.py`):

- `POST /api/search/dive` — launches a deep-dive execution.
- `POST /api/configurations` — used by the `Save Configuration` modal.
- `GET /api/repositories/catalog` and `GET /api/credentials` — consumed indirectly through `useRepoCatalog`; return the static repository catalog and a `{name: {present: bool}}` presence map respectively.

Progress streaming for the launched execution is handled by the shared `/api/executions/{id}/progress` SSE endpoint consumed by `ExecutionContext`; it is not invoked directly from this page.

Request and Response Patterns

`POST /api/search/dive` — body validated by the `DiveRequest` pydantic model:

```
repository: str
query: str
keywords: Optional[list[str]] = None
date_from: Optional[str] = None
date_to: Optional[str] = None
max_results: int = 100
ai_enabled: bool = False
ai_settings: Optional[dict] = None
ephemeral_credentials: Optional[dict[str, str]] = None
saved_configuration_id: Optional[int] = None
```

Handler flow (`search_dive`):

1. `_reject_if_at_manual_cap()` — calls `admission.try_admit(kind="manual")`. On failure raises `HTTPException(429, ..., headers={"Retry-After": "5"})` with a message naming `admission.max()`.
2. Constructs a `SweepEngine` with `config={"ai_enabled": body.ai_enabled, "ai_settings": body.ai_settings}`.
3. Builds `query_params = { query, keywords, date_from, date_to, max_results }` and calls `engine.prepare_execution("deep_dive", [body.repository], query_params)` to insert the pending execution row and return `exec_id`.
4. `progress_store.register(exec_id)` reserves the in-memory progress stream.
5. `_launch_execution(engine, exec_id, conn, ephemeral_credentials=body.ephemeral_credentials)` starts a daemon thread that runs the pipeline.
6. Returns `{"execution_id": exec_id}` immediately — the HTTP call does not block on the pipeline.

`POST /api/configurations` — body validated by `ConfigCreate` (`name: str`, `config_type: str`, `parameters: dict`). For Deep Dive saves, `config_type` is `"manual_dive"` and `parameters` matches the shape described in *Primary User Flows*.

Persistence Touchpoints

- **`executions` table.** `engine.prepare_execution("deep_dive", ...)` inserts a row tagged with `execution_type="deep_dive"`, the repository list, and the `query_params` JSON; the background thread updates `status`, counts, and timestamps through `SweepEngine.run_prepared`. The `saved_configuration_id` from the request body is forwarded to `prepare_execution` and persisted on the `executions` row, which is what powers the `Saved as <name>` badges and the Name column on the Dashboard, Results & Logs, and Calendar popover surfaces.
- **`saved_configurations` table.** `POST /api/configurations` writes a `manual_dive` row that the inline `ConfigLoader` later surfaces.
- **`app_settings` table.** Read-only from this path: `_load_ai_settings_from_db(conn)` pulls every `ai_*` key in `_AI_SETTING_KEYS`. The per-execution `ai_settings` override wins via `merged = {**persisted, **override}` in `_apply_ai_settings_to_engine`.
- **OS keyring.** Stored repository credentials are resolved by the API clients via `get_credential(name)`. No dive endpoint writes to the keyring.
- **Ephemeral-credential store.** `_launch_execution` calls `push_ephemeral(exec_id, ephemeral_credentials or {})` before `engine.run_prepared` and `pop_ephemeral(exec_id)` in `finally`, so per-run keys exist only for the lifetime of the thread and are never persisted.

- **Progress events.** Per-execution events live in `progress_store` during the run; on completion the handler saves them via `save_progress_events(conn, exec_id, events)` into the database and calls `progress_store.cleanup(exec_id)` .

Execution Side Effects

- **Background thread.** `_launch_execution` spawns `threading.Thread(target=_run, daemon=True, name=f"exec-{exec_id}")` . The HTTP request returns as soon as the thread is started.
- **Admission accounting.** `admission.note_admitted(exec_id)` is called at thread start and `admission.note_finished(exec_id)` in the `finally` ; the cap applies across dives, sweeps, and routine fires.
- **LLM client construction.** `_apply_ai_settings_to_engine` merges persisted and override settings, calls `build_llm_client_from_settings(merged, ephemeral=...)` , and populates `engine.config["ai_prompt_params"]` via `_build_prompt_params(merged)` . If the factory returns `None` while `ai_enabled` is true, or raises `ValueError` (e.g. insecure custom base URL), a single `log_entry` progress event is emitted (`"AI skipped: provider not configured"` , `"AI skipped: API key missing"` , or `"AI skipped: <ValueError message>"`) and the execution continues without LLM summarization.
- **Progress / SSE.** Every stage of `SweepEngine.run_prepared` emits typed progress events; the Monitor page, floating widget, and Results & Logs Progress tab all attach via SSE. On completion the in-memory stream is flushed into the database so historical runs replay the same events.
- **Routine email hook.** The `_launch_execution` `finally` block includes the IMPL-R7 routine-completion email hook, but it is gated on `execution_type == "automated_sweep"` and a non-null `routine_id` ; it never fires for a `deep_dive` execution.
- **No direct email or cloud sync.** Deep Dive itself does not trigger email dispatch, Google Drive upload, or cloud-sync writes. Any downstream artifact handling (reports, figures, logs) is owned by `SweepEngine` and the reporting module it invokes.